EXERCISE 4 · AFTER DECK 2 · ANSWER KEY

Guardrails, Strands, and End-to-End Agent Design

Synthesis — design a working agent and reason about what can go wrong.

6 questions · For trainer use · Distribute after the exercise

HOW TO USE
Walk through each answer with the cohort. The explanation is what matters — many of these have answers that look right but fail for subtle reasons.

Q1 OF 6 · CONCEPT

True or false: when you use the Strands Agents SDK, it bypasses the Converse API and uses a different, more efficient protocol to talk to Bedrock.

- **A.** True — Strands uses a custom binary protocol for lower latency.
- **B.** False — Strands calls the same Converse API underneath; it just orchestrates the loop for you.
- **C.** True — Strands talks directly to the model server, skipping Bedrock entirely.
- **D.** False — Strands only works with Anthropic models, not Bedrock.

ANSWER

B.

This is the central pedagogical point of doing chapters 1-7 the hard way. Strands is a layer; the engine underneath is still Converse. Every `agent(user_input)` *call may translate to one or more Converse calls. When the agent does something weird, you debug by inspecting the Converse calls it made.*

Q2 OF 6 · BUG SPOT

A team configured a Guardrail with strict content filters. Their app started blocking legitimate questions. They want to relax the filters for testing. What is the **wrong** way to do this?

- **A.** Create a new version of the guardrail with looser thresholds and point dev to the new version.
- **B.** Edit the existing guardrail's thresholds in place and keep using the same version number.
- **C.** Configure two guardrails — one strict for prod, one loose for dev — and reference different IDs per environment.
- **D.** Leave the guardrail unchanged and disable it in dev by not passing the guardrail config.

ANSWER

B (the wrong approach).

Versions exist precisely to prevent in-place edits to active guardrails. If you edit and reuse the same version number, production may see the new behavior before it's tested. The right pattern is: create a new version → roll it forward deliberately → keep the old version available for rollback. A, C, and D are all valid alternatives.

Q3 OF 6 · BUG SPOT

Spot the issue in this Strands agent setup. The agent runs but never seems to use the knowledge base for any query.

```
from strands import Agent
from strands.models import BedrockModel
from strands_tools import retrieve

# KB ID not set anywhere

model = BedrockModel(
    model_id="us.amazon.nova-lite-v1:0",
    region_name="us-east-1",
)

agent = Agent(
    model=model,
    tools=[retrieve],
    system_prompt="You are a helpful assistant.",
)
```

ANSWER

Two issues: (1) `KNOWLEDGE_BASE_ID` environment variable is never set; (2) the system prompt gives the model no reason to use the retrieve tool.

The built-in `retrieve` *tool reads* `os.environ["KNOWLEDGE_BASE_ID"]` *to know which KB to query. Without it, calls fail silently or return nothing. Separately, the system prompt should explicitly tell the agent when to use retrieve ("use the retrieve tool to search internal docs for policy questions"). Without that guidance, the model often answers from training data instead.*

Q4 OF 6 · CONCEPT

Looking at the before/after collapse table in Chapter 8, which of the following is **NOT** a thing the Strands Agent class handles for you?

- **A.** Appending the model's response to the conversation history
- **B.** Dispatching tool calls to the right Python function
- **C.** Calling Bedrock from a region your code doesn't have access to
- **D.** Applying the guardrail to every model call

ANSWER

C.

Strands does not magically grant your code access to AWS regions or accounts you can't already call. It runs as your IAM principal, in your network, with your credentials. The other three (A, B, D) are exactly what the Agent class manages for you.

Q5 OF 6 · CONCEPT

You see this pattern in production code: the same Guardrail ID is referenced from a plain Converse call, a Knowledge Base query, *and* a Strands agent's BedrockModel. What's the implication?

A. Bad practice — guardrails should be unique per API surface.

B. Good practice — one guardrail, configured once, enforced everywhere; safety stays consistent across the application.

C. Will throw an error — guardrails can only attach to one call type at a time.

D. Will silently fall back to no guardrail on whichever call ran second.

ANSWER

B.

This is the whole point of Bedrock Guardrails. You define a safety policy once and apply it across every Bedrock entry point — Converse, RAG, agent. The alternative (per-surface filters) means you re-implement and re-test safety logic every time you add a new component.

Q6 OF 6 · DESIGN SCENARIO

You're asked to build an agent for an HR team. It should answer policy questions (leave, expenses, benefits) from internal docs, and let employees check their own remaining vacation days via a backend API.

Sketch the agent in 3-5 lines: which tools, which guardrails, and one risk you'd flag to the security team.

ANSWER

Strong sketches include:

***Tools**: built-in* `retrieve` *against a KB indexing the policy docs, plus a custom* `@tool` *like* `get_vacation_balance(employee_id)` *that calls the HR API.* ***Guardrails**: PII filter (mask SSNs and home addresses if they leak in), denied topics (legal advice, medical advice), content filters at moderate threshold.* ***Risk to flag**: the* `get_vacation_balance` *tool must validate that the requesting user matches* `employee_id` *— otherwise prompt injection could trick the agent into reading someone else's balance. The agent itself can't enforce authorization; your tool code must.*